CMPSC 448 Midterm Project: Identifying LLMs from Their Responses

Date: October 11, 2026

Team lead: Adithya Vemulapali

Group members: Shreya Garg, Ani Ganesan

Repository: https://github.com/adithyavemulapalli/448-midterm-project.git

A small CNN can tell which of three LLMs wrote a response with 86.0% test accuracy; an LSTM reaches 60.7%, against 33.3% chance. We built a balanced dataset of 1,500 responses: the same 500 prompts answered by Llama 3.1 8B, Gemini 3.8 Flash, and Qwen 2.5 7B. Classifiers that see only the prompt score exactly 33.3%, because every prompt is shared by all three models. Adding the prompt to the response raises accuracy to 90.0% (CNN) and 68.0% (LSTM). For extra credit, we tested transfer to unseen writing tasks (RQ3) and which signals the models use (RQ4). Accuracy drops on unseen tasks, and formatting such as punctuation and line breaks carries much of the signal.

1. Problem Definition & Dataset Curation

Task. Given a response written by an LLM, predict which of three LLM families wrote it. This is closed-set, three-class text classification. The input is the response text (and, for RQ2, the user's prompt); the label is the model family.

Data format. Every record is a (LLM_name, LLM_Input, LLM_output) triple: `model_family` / `model_name`, `prompt`, and `output`. Records also store `prompt_id`, `prompt_category`, `temperature`, `max_tokens`, and `timestamp_utc`. Only the text fields are used as classifier input; metadata is never fed to the models. Throughout this report, a word is a run of letters or digits (punctuation is not counted).

Family (label)	Model	How collected	Responses	Median words
llama	llama3.1:8b	Ollama, `collect_ollama_remote.py`	500	343
gemini	gemini-3.8-flash	Gemini API, `collect_llm_outputs.py`	500	312
qwen	qwen2.5:7b (Q4_K_M)	Ollama, `collect_ollama_remote.py`	500	327.5
Total	3 families		**1,500**	

Prompts. llm_classification_prompts_500.json holds 500 unique prompts, 25 in each of 20 categories: explanations, creative fiction, poetry, humor, how-to, recipes, code generation, code debugging, math, riddles, summarization, translation, opinions/debate, marketing copy, business writing, history, science, philosophy/ethics, advice, and travel. Each model answered the exact same 500 prompts, so classes are perfectly balanced (500/500/500).

Collection. We generated every response ourselves in September–October 2026. All requests used temperature 0.7 and a 512-token limit. Llama and Qwen ran locally through Ollama; Gemini used the Google AI Studio API with thinking requested disabled; thought-token metadata was not retained to verify enforcement. To finish Qwen faster, we split its run across a Mac (M2 Pro) and a PC (RTX 4070) serving the same model file. Collection scripts save after every response, so interrupted runs resumed without losing data.

Data quality checks. All 1,500 records have non-empty outputs, matching prompt text and category, and no duplicate prompts or exact duplicate outputs. Two known issues were kept as-is and noted: Qwen response #84 mentions its own name (we mask model names during preprocessing), and Gemini response #51 is an incomplete two-line haiku. We also dropped a fourth candidate dataset (GPT-OSS via a free Pollinations endpoint) because its outputs contained provider advertisements and unclear provenance.

Rights and privacy. The dataset contains only our prompts and the models' generated answers. It includes no personal, private, confidential, or third-party data, no scraped conversations, and no API keys (keys are read from environment variables only).

Limitations. The 512-token cap does not equalize length: tokenizers differ, and some responses are cut off mid-sentence. Each family is represented by one model and one generation setup, so results describe these three models, not every model in each family.

2. CNN/RNN Implementation & Training

Both models are written in PyTorch in train_authorship_attribution.py and share the same preprocessing, split, and training loop, so differences come from the architecture.

Preprocessing and tokenization. Text is lowercased and split into words, punctuation marks, and newlines (each kept as its own token), with full Unicode support for non-English text. Model names such as "Qwen", "Gemini", "Llama", or "GPT" are replaced by a <model> token so the classifier cannot simply read the answer. Responses are cut to the first 256 tokens and prompts to the first 64. For input + output, the input is prompt + <sep> + response, so adding the prompt never removes response tokens. The vocabulary is built from training data only (tokens seen at least twice); unseen words map to <unk>.

Split. We split by prompt, not by response: all three answers to a prompt stay in the same set, so test prompts are never seen during training. Each of the 20 categories gives 20 prompts to training and the other 5 to validation and test. That yields 400 / 50 / 50 prompts, or 1,200 / 150 / 150 responses, with every set balanced across families and categories. The split uses a fixed seed (42) and is saved with each run.

	CNN	RNN (LSTM)
Embedding	128-d, trained from scratch	128-d, trained from scratch
Encoder	1-D convolutions, 64 filters each of width 3, 4, 5; ReLU	1 unidirectional LSTM layer, 128 hidden units
Pooling / readout	Max-pool over time (padding masked) → 192-d vector	Hidden state at the last real (non-padding) token
Head	Dropout 0.4 → linear layer → 3 classes	Dropout 0.4 → linear layer → 3 classes
Parameters (output-only)	1,313,283	1,346,691

Training. Both models use cross-entropy loss and the AdamW optimizer (learning rate 0.001, weight decay 0.01), batch size 32, and gradient clipping at 1.0. Each trains for up to 15 epochs with early stopping: training stops after 4 epochs without improvement in validation loss, and the weights from the best validation epoch are restored before testing. We did no hyperparameter search and never used the test set to choose a model. All 14 runs trained on an Apple M2 Pro GPU (MPS), taking under 2 minutes in total.

Experiment grid (14 runs). For each of CNN and LSTM: input-only, output-only, and input + output (seed 42); output-only repeated with seeds 7 and 123; an output-only run with punctuation and newlines removed (RQ4); and a held-out-domain run (RQ3).

Extra classifiers. As diagnostic baselines for RQ4, we trained two logistic-regression models (scikit-learn, standardized features): one on word count alone, and one on 11 hand-made style features (words, characters, lines, blank lines, bullet lines, headings, backticks, bold markers, type-token ratio, average word length, punctuation fraction).

Correctness checks. Unit tests in `tests/test_pipeline.py` confirm that splits are balanced and disjoint, that Unicode and name masking work, and that padding does not change either model's prediction.

3. Results & Presentation

All numbers are on the held-out test set: 150 responses (50 prompts × 3 families). Chance is 33.3%. The 95% intervals come from bootstrap resampling of whole test prompts (2,000 draws).

RQ1: Can we identify which LLM generated a response?

Yes. From the response alone, the CNN is right 86.0% of the time and the LSTM 60.7%, both well above chance.

Model (output only)	Accuracy [95% CI]	Macro-F1	Gemini recall	Llama recall	Qwen recall	Best epoch
CNN	86.0% [79.3–92.0]	0.859	96%	78%	84%	15
LSTM	60.7% [52.7–68.7]	0.601	78%	40%	64%	6

Confusion matrices (rows = true family, columns = predicted; 50 responses per row):

CNN	→ Gemini	→ Llama	→ Qwen
Gemini	**48**	0	2
Llama	2	**39**	9
Qwen	5	3	**42**

LSTM	→ Gemini	→ Llama	→ Qwen
Gemini	**39**	3	8
Llama	9	**20**	21
Qwen	3	15	**32**

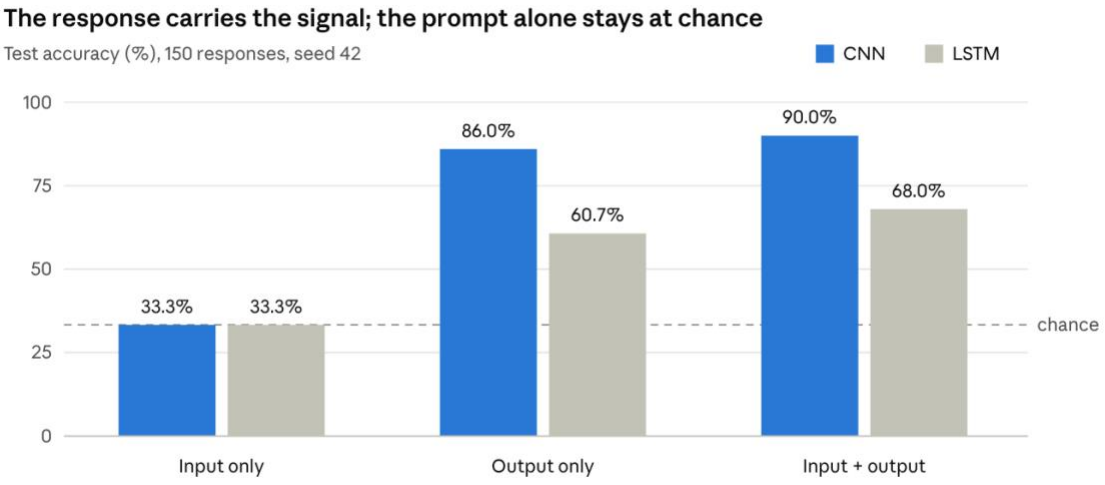
Gemini is the easiest family to recognize for both models. Most mistakes are Llama and Qwen being confused with each other: both are open-weight 7–8B models run through Ollama, and their formatting is similar (see RQ4).

Stability across random seeds (output only, same split):

Model	Seed 42	Seed 7	Seed 123	Mean ± SD
CNN	86.0%	88.0%	88.7%	87.6 ± 1.4
LSTM	60.7%	66.7%	52.7%	60.0 ± 7.0

The CNN is consistent; the LSTM swings by 14 points depending on initialization. Seed 42 was fixed in advance, not picked as the best run.

RQ2: Does the user's prompt help identify the LLM?



The prompt alone cannot identify the model, while the response can. Adding the prompt to the response helps a little: +4.0 points for the CNN (paired 95% CI +1.3 to +7.3) and +7.3 points for the LSTM (CI 0.0 to +15.3, so not clearly significant). Section 4 explains why input-only is exactly 33.3%.

4. In-Depth Analyses & Experiments

RQ2 discussion: why input-only is exactly chance

Can the LLM be predicted from the prompt alone? Not in our data, by design. Every prompt was sent to all three models, so each test prompt appears three times with three different labels. A trained model gives the same prediction for identical input, so it is right on exactly one of the three, which is 33.3%. Both models hit that value exactly. This matches the expected prompt-only control; separate split and preprocessing checks address other leakage risks.

Does including the input help? Slightly: +4.0 points for the CNN and +7.3 for the LSTM. The prompt may help the classifier interpret task-dependent response patterns; our experiments do not establish the cause. The CNN gain's paired 95% interval excludes zero; the LSTM's does not. These intervals reflect sampling of test prompts, not variability across training seeds.

If input-only had worked well, why? It would mean the prompts themselves differ by model. That happens in datasets where each model's responses were collected from different sources, users, or topics: the classifier then learns the topic, not the model. A high input-only score is a red flag for this kind of bias, and is why we gave all models identical prompts.

RQ3 (extra credit): Do fingerprints generalize across tasks?

Partly. We held out four writing categories (creative fiction, poetry, marketing copy, business writing: 100 prompts, 300 responses) and trained new output-only models on the other 16

categories (320 / 40 / 40 prompts). Vocabulary and early stopping used only the 16 training categories.

Model	Seen-task test	Unseen writing	Drop	Unseen Gemini	Unseen Llama	Unseen Qwen
CNN	92.5%	64.3% [59.3–69.0]	−28.2 pts	84%	75%	34%
LSTM	64.2%	50.3% [45.7–55.0]	−13.8 pts	69%	41%	41%

- **Performance drops on unseen tasks**, by 28 points for the CNN, but both models stay above the 33.3% chance level.
- **Gemini is the most consistently identifiable** family across domains (84% and 69% recall on unseen writing). Qwen is the hardest to carry over for the CNN (34%).
- **The classifier learns a mix.** Some signal is a general fingerprint (it transfers), but a large part is tied to the task mix seen in training. Our results show the drop, not its exact cause; collection order and response length may also contribute.

RQ4 (extra credit): What characteristics distinguish the LLMs?

In our analyses, formatting and vocabulary habits separate the families more clearly than length does. Median values over all 500 responses per family (words counted as in Section 1):

Feature (median)	Gemini	Llama	Qwen
Words	312	343	327.5
Type-token ratio (lexical diversity)	**0.612**	0.486	0.511
Punctuation share of characters	**0.080**	0.057	0.051
Markdown heading lines	**2**	0	0
Bold markers (**)	14	12	10
Bullet lines	5	7	6

Gemini stands out: it uses more varied vocabulary, more punctuation, and markdown headings, while Llama and Qwen look alike. This matches the confusion matrices, where most errors are Llama ↔ Qwen.

Diagnostic baselines (logistic regression, same test split):

Classifier	Test accuracy	Macro-F1
Word count only	33.3%	0.259
11 style features	72.0%	0.712

Our word-count-only logistic-regression baseline scored chance on this test set, while 11 simple style features reach 72%. That baseline does not rule out nonlinear length effects or interactions between length and other features. The largest weights are on lexical diversity, punctuation share, and heading lines.

Ablation: removing formatting. We retrained both networks with punctuation and newlines deleted (text was already lowercased and model names masked):

Model	Normal text	No punctuation / newlines	Change [paired 95% CI]
CNN	86.0%	66.7%	−19.3 pts [−26.7, −12.0]
LSTM	60.7%	45.3%	−15.3 pts [−25.3, −5.3]

Removing formatting substantially reduces performance, so the signals are **both structural and linguistic**: structure (punctuation, line breaks, markdown) accounts for a large share, but the models still beat chance on plain words alone. One caveat is that removing punctuation also fits more words into the 256-token window, so the drop is not purely due to punctuation.

5. Lessons & Experience Learned

What worked

- **Using identical prompts for every model.** It kept classes balanced and turned the input-only experiment into a built-in control: any score other than 33.3% would have pointed to a bug.
- **Splitting by prompt, not by response.** Keeping all three answers to a prompt together stopped the model from scoring well just by recognizing a prompt it had already seen.
- **A simple CNN.** It trained in seconds, was stable across seeds (87.6% ± 1.4), and beat the LSTM by about 25 points.
- **Checkpointed data collection.** Saving after every response meant crashes and machine switches never lost data.

What did not work as well

- **The LSTM.** It was weaker and unstable (52.7% to 66.7% across seeds). A unidirectional LSTM reading 256 tokens must squeeze everything into its final hidden state, while the CNN's max-pooling picks up short local patterns (punctuation, markdown, common phrases) from anywhere in the text.

- **Our first training prototype.** It had three problems we only found on review: its ASCII-only tokenizer dropped non-English characters, its 128-token limit cut off 1,276 of 1,500 responses, and its random split left math out of validation and code debugging out of test. We rewrote the pipeline with Unicode tokenization, a 256-token window, and a category-balanced split.
- **A fourth model family.** We collected GPT-OSS responses from a free endpoint but dropped them: many outputs contained provider advertisements or reasoning wrappers, which would have made that class trivially easy to spot.

Challenges

- **Collection speed.** Running Qwen locally one prompt at a time was slow. We first ran two requests at once on the Mac, then moved to a PC with an RTX 4070, and finally used both machines together with one shared writer so no response was duplicated or lost.
- **Unequal lengths.** A 512-token limit does not mean equal-length answers, because each model's tokenizer counts differently and some answers stop early. We checked that length alone could not explain our results (the word-count baseline scored chance).
- **Identity leaks.** One Qwen response named itself. We masked model names in all text so the classifier had to learn style, not names.

What we would do differently

- Record each response's finish reason and token count for every model from the start; we only have them for part of the Qwen data.
- Add more model families and more than one model per family, so results describe families rather than single models.
- Train the CNN longer: its best epoch was the last allowed (15), so it may not have finished improving.
- Try a bidirectional LSTM or pretrained embeddings, and repeat the RQ3 domain experiment with several seeds.

AI Tool Use and Reproducibility

AI assistance. As the assignment permits, we used AI coding assistants (OpenAI Codex and Anthropic Claude) to help write data-collection and training code, debug, and organize this report. The team reviewed the code and results and is responsible for everything submitted.

Reproduce. All code, data, per-run metrics, predictions, and the two main trained models are in the repository.

```
pip install -r requirements.txt
python -m unittest discover -s tests -v
python run_experiments.py --device auto
python analyze_results.py
```